BRACT's
Vishwakarma Institute of Information Technology, Pune
Practical Implementation Sheet

Department: IT	**Semester:** 6	**Academic Year:** 2025-26	**Assignment no.:**
Class/ Division/ Batch: TY(B)-B2			
Course: TY Project		**Name of Student**: Dhage Samarth Harish	

🧠 1. System Architecture (Project Structure)

Overall Flow

Frontend (React / Web App)
↓
Node.js Backend (Express API)
↓
MySQL Database
↓
ML Microservice (FastAPI)
↓
Random Forest Model

Flow Explained

User App मध्ये:

User Signup
↓
User Login
↓
Financial Profile Submit
↓
Behavioral Questionnaire
↓
Backend Feature Vector बनवतो
↓
ML Model ला data पाठवतो
↓
ML Model Panic Probability देतो
↓
Anti-Sell AI Recommendation देतो
↓
Crash Simulation दाखवतो

🌐 Base URL

http://localhost:5000

Frontend calls will look like:

http://localhost:5000/api/....

🔐 Authentication APIs

These APIs handle **user registration and login**.

1️⃣ Signup API

Endpoint

POST /api/auth/signup

Purpose

नवीन user register करण्यासाठी.

Request Body

```
{
 "fullName": "Samarth Dhage",
 "email": "samarth@test.com",
 "phone": "9876543210",
 "password": "123456"
}
```

Response

```
{
 "message": "User created successfully"
}
```

Frontend Integration

React मध्ये:

axios.post("http://localhost:5000/api/auth/signup", data)

2️⃣ Login API

Endpoint

POST /api/auth/login

Purpose

User login करतो आणि **JWT token मिळतो**.

हा token पुढील APIs साठी लागतो.

Request Body

```
{
 "email":"samarth@test.com",
 "password":"123456"
}
```

Response

```
{
 "message":"Login successful",
 "token":"JWT_TOKEN",
 "user":{
  "id":1,
  "fullName":"Samarth Dhage",
  "role":"user"
 }
}
```

Important

Frontend ने **token save करणे आवश्यक आहे**.

Example:

localStorage.setItem("token", token)

🔑 **Authorization (Important)**

Next APIs साठी header लागतो:

Authorization: Bearer TOKEN

Example

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

👤 User APIs

3️⃣ Financial Profile API

Endpoint

POST /api/user/financial-profile

Purpose

User चे **financial data store करणे**.

ML model ला ही माहिती लागते.

Headers

Authorization: Bearer TOKEN

Request Body

```
{
 "age":30,
 "income":800000,
 "portfolio_value":400000,
 "past_loss_percent":10,
 "panic_history":0
}
```

Meaning of Fields

Field	Meaning
age	user age
income	yearly income
portfolio_value	current investment value
past_loss_percent	previous loss percentage
panic_history	panic selling history

Response

```
{
 "message":"Financial profile stored successfully"
}
```

Frontend Integration

```
axios.post("/api/user/financial-profile",data,{
 headers:{
  Authorization:`Bearer ${token}`
 }
})
```

📊 4️⃣ Behavioral Assessment API

Endpoint

POST /api/user/assessment

Purpose

User answers questionnaire.

Backend:

Behavioral score calculate करतो
↓
ML model ला data पाठवतो
↓
Panic probability मिळते
↓
Portfolio recommendation देतो

Headers

Authorization: Bearer TOKEN

Request Body

```
{
 "answers":[5,6,7,4,6,5,6,7,5]
}
```

Response Example


```
{
 "message":"Assessment completed",
 "scores":{
  "emotional_score":18,
  "risk_score":17,
  "financial_score":15,
  "horizon":5
 },
 "mlOutput":{
  "panic_probability":0.32,
  "regret_probability":0
 },
 "recommendation":"Moderate Portfolio"
}
```

Frontend Usage

Frontend:

User questionnaire answers submit करेल

Then show:

Recommended portfolio

📉 5️⃣ Crash Simulation API

Endpoint

POST /api/user/simulate

Purpose

Market crash simulation दाखवणे.

User ला कळते:

10% crash
20% crash
30% crash

Headers

Authorization: Bearer TOKEN

Request Body


```
{
 "portfolio_value":500000
}
```

Response


```
{
 "message":"Crash simulation complete",
 "crash_10_percent":450000,
 "crash_20_percent":400000,
 "crash_30_percent":350000
}
```

Frontend Idea

Show graph:

Portfolio value vs crash

👨‍💼 6️⃣ Admin Analytics API

Endpoint

GET /api/admin/analytics

Purpose

Admin dashboard साठी analytics.

Headers

Authorization: Bearer ADMIN_TOKEN

Response


```
{
 "total_users":10,
 "average_risk_score":6.2,
```

```
 "average_panic_probability":0.41
}
```

Frontend Dashboard

Admin panel मध्ये:

Total users
Average risk score
Average panic probability

🤖 ML Integration Architecture

Backend ML call:

```
Node.js
↓
POST http://127.0.0.1:8000/predict
↓
FastAPI
↓
Random Forest Model
```

ML input:

```
{
 "age":30,
 "income":800000,
 "portfolio_value":400000,
 "emotional_score":18,
 "risk_score":17,
 "financial_score":15,
 "horizon":5,
 "past_loss_percent":10,
 "panic_history":0
}
```

📦 Database Tables

users

user information

financial_profiles

financial user data

assessments

behavior + ML prediction

model_logs

ML monitoring

🧩 Frontend Integration Summary

Frontend should call APIs in this order:

1 Signup
2 Login
3 Save Token
4 Financial Profile
5 Assessment
6 Show Recommendation
7 Crash Simulation

🏁 Final System

User
↓
Frontend
↓
Node Backend
↓
MySQL
↓
FastAPI ML Server
↓
Random Forest Model
↓
Investment Recommendation

🎉 Backend Status

Authentication ✓
Financial Profile ✓
Behavioral Engine ✓
ML Integration ✓
Recommendation Engine ✓

Crash Simulation ✓
Admin Analytics ✓

Backend Completion

100%